Practical-1

Create Amazon AWS/Microsoft Azure/Google Cloud account

Core Theory

Term	Meaning	Purpose
Cloud Computing	Using internet-based resources instead of own hardware	Scalability, lower cost
AWS	Amazon's cloud platform	Provides cloud services
EC2	Virtual machine service	Run apps/servers
S3	Cloud object storage	Store files/websites
VPC	Private cloud network	Networking/security
IAM	User & permission management	Access control
AMI	Preconfigured OS template	Launch instances quickly
Instance	Running virtual machine	Cloud server
Security Group	Firewall for EC2	Controls traffic
Subnet	Smaller network inside VPC	Network separation
Internet Gateway	Connects VPC to internet	Internet access
Route Table	Traffic routing rules	Packet direction
Policy	Permission rules in IAM	Defines access
Role	Temporary permissions	Secure service access
Free Tier	Limited free AWS usage	Learning/testing

Public vs Private Cloud

Public Cloud	Private Cloud
Shared infrastructure	Dedicated infrastructure
AWS/Azure/GCP	Organization-owned

Advantages / Disadvantages

Advantages	Disadvantages
Scalable	Internet dependency
Low upfront cost	Security concerns
Accessible anywhere	Recurring cost
Easy maintenance	Vendor lock-in

Practical-2

EC2->Launch Instance->Any name->Default AMI->Default Instance->Create new key pair(default=>RSA, .pem)->Default Network settings->Default Storage->Launch Instance

open linux terminal(for ssh):-

chmod 400 ~/mykey.pem
ssh -i ~/key.pem ec2-user@(ipv4 of ec2)

Practical-3

Main Idea:-

```
AWS Network
 ├── Public Subnet
 │      └── EC2 with internet
 │
 └── Private Subnet
        └── EC2 without direct internet
```

VPC->Create VPC->VPC and More->Default Everything(1 availability zone, 1 public and 1 priv subnet, 1 in AZ NAT gateways, no VPC endpoints)->Create VPC

After VPC Creation

Next steps are:

1. Launch Public EC2

- Public subnet
- Auto-assign public IP = ENABLED

Purpose:
direct SSH/internet access.

---

2. Launch Private EC2

- Private subnet
- Auto-assign public IP = DISABLED

Purpose:
internal/private server.

in ec2, while creating subnets, select the vpc, select the public/private subnet respectively, auto assign enabled for public and disabled for private

then ssh into public
ssh -i key.pem ec2-user@PUBLIC_EC2_IP

then open a new terminal and copy the key
scp -i key.pem key.pem ec2-user@PUBLIC_EC2_IP:~(rename the key).pem

then back to the public ssh terminal, ssh into private from there
ssh -i (renamedkey).pem ec2-user@PRIVATE_EC2_IP

Practical-4

1. Create 2 ec2(tick allow http traffic) and ssh into both then install apache

sudo yum install httpd -y
sudo systemctl start httpd
echo "Server 1" | sudo tee /var/www/html/index.html

do server 2 with everything else constant for 2nd ec2

2. Target Groups(Part of EC2)->Create->Instances Type->Register Targets(select both ec2s and include as pending below)->Create

3. Search Load Balancers(Part of EC2)->Create Load Balancer->Application Load Balancer->Internet Facing->select any 2 availability zones->select target group->create

4. the sg group that the load balancer uses must have inbound rule http added, if not then add it

5. wait till status goes from provisioning to active and then do http:/dnsOfLoadBalancer(refresh to show load balancing)

Practical-5

1. Create index.html on your pc
(<h1>Hello CC</h1>)

2. Create S3 Bucket(Untick Block Public Access) and upload index.html

3. Bucket Properties->Static Website Hosting(index document=index.html)

4. Bucket Permissions->Bucket Policy->

```
{
"Version":"2012-10-17",
"Statement":[{
"Effect":"Allow",
"Principal":"*",
"Action":"s3:GetObject",
"Resource":"arn:aws:s3:::YOUR_BUCKET_NAME/*"
}]
}
```

5. Bucket Properties->Website endpoint

Practical-6

1. Launch one ec2

2. CloudWatch->All Metrics->EC2->Per Instance Metrics->CPUUtilization(Pick your EC2)->Create Alarm->put a threshold like “than... 15”->Notification(create new SNS topic->put a temp mail->open mail and confirm subscription)->Create Alarm

3. ssh into ec2 and type: “yes > /dev/null” to stress cpu, ctrl+c to stop[should get an alarm notification in mail]

Practical-7

openstack is too big to download(minimum 2-4gb to maximum 10+gb) so just write steps:-

sudo snap install microstack
sudo microstack init --auto

Practical-8

(Also won’t work since has to be done using openstack)

http://IP/dashboard(would open horizon dashboard IF openstack was installed)
admin/admin(login)
Identify->Projects->Create Project(Create Project)
Compute->Images->Create Image(Upload Image)
Admin->Flavors->Create Flavor(Create Flavor, eg. 1vCPU,1GB ram,10GB disk)
Compute->Instances->Launch Instances(Launch Instance, select image, flavor, network and launch vm)

Practical-9

(again openstack so won't work)

http://IP/dashboard(would open horizon dashboard IF openstack was installed)
admin/admin(login)

Network->Create Network->private-network->subnet(like 192.168.1.0/24)
Network->Create Network->public-network->enable External Network

Network->Routers->Create Router->Set Gateway: public-network
Routers->Add Interface: private subnet

Practical-10

Term	Meaning
IAM	Identity and Access Management
Policy	Permission rules
Role	Set of permissions
Least Privilege	Give minimum required access
Encryption	Protecting data using encoding
Firewall Rule	Controls network traffic
Security Group	AWS virtual firewall

Steps

1. Create IAM User

AWS →

IAM → Users → Create User

Assign password/access.

---

2. Create IAM Policy

IAM →

Policies → Create Policy

Example:
Allow:

- only S3 read access

3. Assign Least-Privilege Role

Attach policy to user.

Give only required permissions.

Example:

```
Read-only S3 access
```

not full admin access.

4. Enable S3 Bucket Encryption

Go:

S3 → Bucket → Properties

Enable:

```
Server-side encryption
```

Choose:

```
AES-256
```

5. Configure Firewall Rule

Go:

EC2 → Security Groups

Edit inbound rules.

Allow:

Type	Port
SSH	22
HTTP	80

6. Test Rule

Try:

- opening website
- SSH connection

Verify allowed traffic works.

Practical	What We Did	Process/Flow	Why
1	Created AWS/Azure/GCP account	Sign up → verify → login to cloud console	To access and use cloud computing services/resources
2	Launched EC2 and connected via SSH	Create EC2 → generate key pair → launch VM → SSH connect	To create and remotely manage a cloud virtual machine
3	Created VPC with public/private subnet	Create VPC → create subnets → configure NAT/IGW → launch EC2s	To isolate networks and securely separate public/private resources
4	Used Load Balancer with 2 EC2s	Launch 2 EC2s → install Apache → create target group → create ALB	To distribute traffic, improve scalability and high availability
5	Hosted static website on S3	Create bucket → upload HTML → enable hosting → add bucket policy	To deploy a low-cost website using cloud object storage
6	Created CloudWatch alarm + SNS	Launch EC2 → monitor CPU metric → create alarm → send SNS alert	To monitor system performance and receive automatic alerts
7	Installed OpenStack/MicroStack	Install MicroStack → initialize services → access Horizon dashboard	To set up a local/private cloud environment for practice
8	Created VM instance in OpenStack	Login Horizon → upload image → create flavor → launch instance	To understand how cloud platforms launch/manage virtual machines
9	Configured OpenStack networking	Create networks → create router → connect networks → assign IP	To allow VM communication and provide internet connectivity
10	Configured IAM/security/encryption	Create IAM user → assign policy → enable encryption → edit SG	To control access, secure data and protect cloud resources